

LakeTemperature Testing Software

Slides by Aidan Bensch and Sam Allen

Background

- We are testing a module of *E3SM*,¹ an environmental model. This module, *LakeTemperature*, is part of the greater *ELM* (*E3SM* Land Model) which was originally a fork of the *CLM* (Community Land Model).
- We are able to test specifically this module using *SPEL*,² a software that adds a *NetCDF*³ interface for the constants, inputs, and outputs of a module of *E3SM* and compiles that specific part of the model. The executable produced by this is named `elmtest.exe`.

1. "Energy Exascale Earth System Model." *E3SM*, U.S. Department of Energy: Office of Science, n.d., e3sm.org/.

2. Schwartz, Peter. "SPEL_OpenACC." *GitHub*, 16 Oct. 2025, github.com/peterdschwartz/SPEL_OpenACC.git.

3. "NetCDF." *NSF UNIDATA*, n.d., www.unidata.ucar.edu/software/netcdf.

Project Goals

- Develop a software that can systematically change the constants used in the LakeTemperature model and allow us to visualize and observe patterns in how this affects its output.
- Apply testing techniques to try and find values for constants that will cause physically impossible or otherwise erroneous outputs. Apply assertions pertaining to the values of constants, inputs, and outputs based around physical laws and other constraints provided by domain scientists.

Our Testing Solution

- Automates the process of modifying constants in spel-constants0001.nc and the process of analyzing the resulting values in fut-outputs0001.nc
- Both the process of choosing values for each constant and the process of analyzing the resulting outputs is extendable through sampling plugins and analysis plugins respectively.

Sampling Plugins

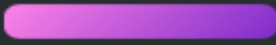
SampleGroup(s)

Testing

Tables of Testing Data

Analysis Plugins

Key

-  = Input Data
-  = Reference Data
-  = Output Data



Testing Expanded



Sample Generation



Using a Sampling Plugin to create samples
with unique values for the 23 scalar constants



Running `elmtest.exe`



Running the program using the constants
and recording the results



Analysis



Using an Analysis Plugin to analyze the results,
especially in comparison to reference data

Constants, Inputs, and Outputs

- The 26 *constants* control how the model behaves and manipulates data.
 - We don't yet support modification of the 3 array constants. Of the other 23 scalar constants we do experiment with, 8 have no observable effect on the output.
- The 25 *inputs* provide starting data for the model.
- The 9 *outputs* constitute data that the model has generated.
- 8 variables are both an input and an output.

Table of Constants

Array Constants	crop, iscft, percrop
Scalar Constants with Observable Outputs	Betavis, cnfac, cpliq, denh2o, grav, hfus, lake_no_ed, lakepuddling, nlevgrnd, nlevlak, nlevsno, tkwat, vkc
Scalar Constants without Observable Outputs	cpice, denice, depthcrit, mixfact, pudz, tkice

Tfrz, tdmx, n2min, and za_lake from LakeTemperature.md not included yet, don't know if their outputs are observable or not

This table uses explicitly the constants mentioned in LakeTemperature.md plus the array constants

Table of Inputs and Outputs

Inputs	col_es%t_grnd, col_pp%dz, col_pp%dz_lake, col_pp%z, col_pp%z_lake, col_pp%zi, col_pp&lakedepth, col_pp&snl, col_ws%h2osno, col_ws%h2osoi_ice, col_ws%h2osoi_liq, lakestate_vars%etal_col, lakestate_vars%ks_col, lakestate_vars%lake_raw_col, lakestate_vars%ws_col, soilstate_vars%csol_col, soilstate_vars%tkmg_col, soilstate_vars%tksat_u_col, soilstate_vars%watsat_col, solarabs_vars%fsds_nir_d_patch, solarabs_vars%fsds_nir_i_patch, solarabs_vars%fsr_nir_d_patch, solarabs_vars%fsr_nir_i_patch, solarabs_vars%sabg_lyr_patch, solarabs_vars%sabg_patch
Both Inputs and Outputs	col_es%t_lake, col_es%t_soisno, lakestate_vars%lake_icefrac_col, lakestate_vars%lake_icethick_col, veg_ef%eflx_gnet veg_ef%eflx_sh_grnd, veg_ef%eflx_sh_tot, veg_ef%eflx_soil_grnd
Outputs	ch4_vars%grnd_ch4_cond_col, col_ef%errsoi, col_es%hc_soi, col_es%hc_soisno, col_wf%qflx_snofrz, col_ws%frac_iceold, lakestate_vars%betaprime_col, lakestate_vars%lakeresist_col, lakestate_vars%savedtke1_col

NetCDF

- Network Common Data Form (*NetCDF*) is a format for self-describing datasets. Each file is composed of “dimensions” and “variables.” Variables are multi-dimensional arrays, with the meaning of each dimension described in the file.

```
netcdf x_times_y {  
  dimensions:  
    x = 5 ;  
    y = 5 ;  
  variables:  
    float x(x) ;  
    float y(y) ;  
    float z(x, y) ;  
  data:  
  
    x = -2, -1, 0, 1, 2 ;  
  
    y = -4, -2, 0, 2, 4 ;  
  
    z =  
      8, 4, -0, -4, -8,  
      4, 2, -0, -2, -4,  
      -0, -0, 0, 0, 0,  
      -4, -2, 0, 2, 4,  
      -8, -4, 0, 4, 8 ;  
}
```

What is a Sample?

- A sample is a set of values for the constants contained in `spel-constants0001.nc`.
- `elmtest.exe` is run once for each sample.

What is a Sample? (Illustration)

Constants (formerly lakeparams.txt)

```
betavis
0.0
za_lake
0.6
n2min
7.4999999999999993E-005
tdmax
277.0
pudz
0.0
mixfact
10.0
depthcrit
25.0
```

Example Sample (JSON used for illustration)

```
{
  "betavis": 0.0,
  "za_lake": 0.6,
  "n2min": 7.4999999999999993E-005,
  "tdmax": 277.0,
  "pudz": 0.0,
  "mixfact": 10.0,
  "depthcrit": 25.0
}
```

What Outputs are Received per Sample?

- Analysis plugins will receive the outputs from the `spe1-outputs0001.nc` file in the form of multidimensional numpy arrays, which range from one to three dimensions.

Outputs Received by Analysis Plugins (JSON for illustration)

```
{  
  "veg_ef%eflx_soil_grnd": [0.0, ...],  
  "veg_ef%eflx_gnet": [[0.0, ...], ...],  
  ...  
}
```

Sample Groups

- Represent a grouping of ordered samples
 - When the program is run on each sample in a sample group, the model data is returned in the same order as the samples were in.
- Analysis plugins can analyze the result of multiple consecutive runs of `elmtest.exe`.
- Having multiple sample groups is simply a convenient way of organizing similar samples.

Sample Group Illustration

Sample Group Data (JSON for illustration)

```
[
  {
    "betavis": 0.0,
    ...
  },
  {
    "betavis": 0.25,
    ...
  },
  {
    "betavis": 0.5,
    ...
  }
]
```

Ordered Output Data (JSON for illustration)

```
[
  {
    "lakestate_vars%betaprime_col": [0.0, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.25, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.5, ...],
    ...
  }
]
```

Plugins

- Our testing framework supports two types of plugins, sampling and analysis. Sampling plugins define values for constants, and analysis plugins view the outputs of the model.
- Two sampling plugins are currently supported:
 - CSV Index Sampling, Order Sampling
- Four analysis plugins are currently supported:
 - Change Detection, Differences, Fault Finding, NetCDF4 Output

Sampling Plugin Overview and Visuals

- Sampling plugins define a subclass of `Sampler`, returning one or more `SampleGroups`.

Interface

```
class Sampler(ABC):  
    @abstractmethod  
    def get_sample_groups(self) -> Mapping[str, SampleGroup]:  
        pass
```

UML Diagram

Current Sampling Plugins

CSV Index Sampling	Creates samples using CSV files generated by NIST's ACTS (Automated Combinatorial Testing for Software) by linearly interpolating between the values for each constant found in the ranges file
Order Sampling	Generates Order(s), a subclass of SampleGroup. Each Order reads instructions from a JSON file to generate samples using linear interpolation

Analysis Plugin Overview

- Analysis plugins can either be “per sample” or “sample group” analysis plugins.
- “Per sample” plugins define a subclass of `PerSampleAnalyzer`, whose method is called each time `elmtest.exe` is run, and which receives the outputs produced by that specific sample.
- “Sample group” plugins define a subclass of `SampleGroupAnalyzer`, whose method is called after all samples in a group are run through `elmtest.exe`, and which receives all outputs from each.

Analysis Plugin Visuals

Interfaces

UML Diagram

```
class PerSampleAnalyzer(ABC):  
    @abstractmethod  
    def analyze_sample_data(  
        self,  
        sample: Sample,  
        reference_data: ModelData,  
        test_data: ModelData,  
    ) -> tuple[str, FileSystemTree]:  
        pass
```

```
class SampleGroupAnalyzer(ABC):  
    @abstractmethod  
    def analyze_sample_data(  
        self,  
        sample_group: SampleGroup,  
        reference_data: ModelData,  
        sample_data: list[ModelData],  
    ) -> tuple[str, FileSystemTree]:  
        pass
```

Current Analysis Plugins

Per Sample Analyzers	Differences	Finds differences between values in the reference data and the current sample's data, recording the indices of the difference, the two differing values, and the difference between them
	Fault Finding	Uses functions defined by the plugin to make assertions about values in the sample data, similar to a unit testing framework but with assertions pertaining to model values rather than specific function behavior
Sample Group Analyzer	Change Detection	Identifies which inputs and outputs differ from the reference
	NetCDF4 Output	Stores each sample group as an individual NetCDF file, to be used for graphing changes based on our samples

Configuration

refers to the configuration files that control the model testing framework and its plugins - look inside the "lake_temp/config" directory, all the .ini files are loaded by different parts of the program and allow it to be adapted to different models/requirements

